TradeX – Technical Documentation

Smart Stock Trading & Portfolio Management Simulation Platform (MERN)

1. Introduction

Project Title

TradeX – Smart Stock Trading and Portfolio Management

Project Type

Full-Stack Web Application (MERN Stack)

Role

Full Stack Developer

Project Description

TradeX is an interactive, full-stack stock trading simulation and virtual portfolio management platform developed using the MERN stack (MongoDB, Express.js, React.js with Vite, and Node.js). The application serves as a risk-free environment designed to bridge the gap between financial learning and practical market experience. Users can securely register, explore comprehensive US stock listings, analyze live real-time prices and historical performance charts, execute simulated paper trading orders (buy and sell), and manage multiple virtual portfolios to test diverse investment strategies.

2. Project Overview

2.1 Purpose

The primary purpose of TradeX is to provide an accessible, realistic, and risk-free platform for stock market practice. By delivering real-time market data, interactive price trends, and automated transaction management, TradeX allows novice and experienced traders to evaluate strategies, monitor portfolio gains and losses, and understand market dynamics without risking actual financial capital.

2.2 Objectives

- **User Security & Authentication:** Provide secure user registration and login using JWT authentication and bcrypt password hashing.
- **Virtual Portfolio Management:** Enable traders to create, customize, and manage multiple virtual portfolios to test distinct trading models.
- **Paper Trading Simulation:** Facilitate instant execution of market buy and sell orders with live balance and holding updates.

- **Real-Time & Historical Market Analytics:** Deliver live US stock price feeds, detailed stock performance trends, and company insights.
- **Transaction Logging:** Maintain an indelible historical log of all buy/sell transactions with precise timestamps and amounts.
- **Administrative Governance:** Provide dedicated admin capabilities to oversee application health, monitor user trading activity, and manage stock listings.
- **Responsive MERN Architecture:** Maintain high performance, modularity, and clean separation of concerns using the MVC design pattern.

2.3 Key Features

- **User Registration & Authentication:** Secure sign-up, JWT-based login, password encryption with bcrypt, and protected personal dashboards.
- **Real-Time Market Data:** Live stock ticker feeds, current prices, market trends, and company profile statistics.
- **Historical Data & Interactive Charts:** Visual chart representations of stock price history for strategic technical analysis.
- **Virtual Portfolios:** Creation and tracking of multiple portfolios with real-time profit and loss (P&L) calculation.
- **Order & Transaction Management:** Execution of buy/sell orders, order status tracking, and complete historical ledger logging.
- **Comprehensive Stock Listings:** Browse and search through extensive US market stock listings by symbol or company name.
- **Admin Control Console:** Centralized administrative dashboard to view system statistics, monitor user accounts, and maintain market stock feeds.
- **Global Context State:** Persistent authentication state and user session management across React components using Context API.

2.4 Technologies Used

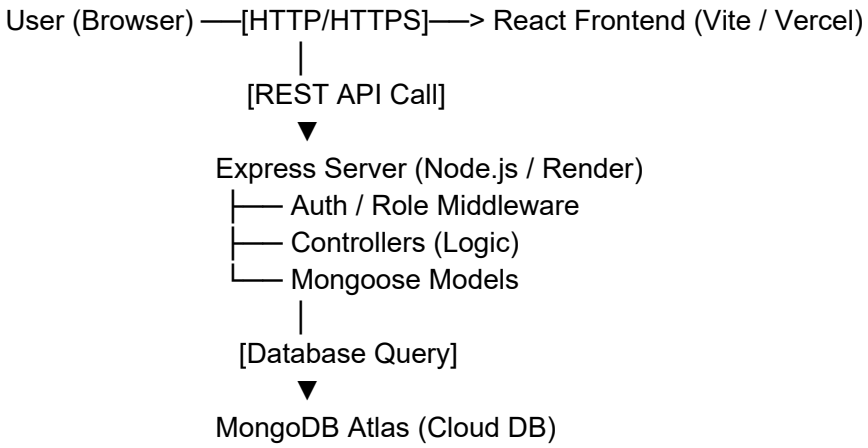
Category	Technology / Tool
Frontend Framework	React.js (v18+)
Build Tool	Vite
Backend Runtime	Node.js
Web Framework	Express.js
Database	MongoDB / MongoDB Atlas
Database ODM	Mongoose

Authentication	JSON Web Token (JWT)
Password Hashing	bcrypt / bcryptjs
HTTP Client	Axios (Pre-configured Instance)
State Management	React Context API (GeneralContext)
API Testing	Postman
Version Control	Git & GitHub
Frontend Hosting	Vercel
Backend Hosting	Render

3. Architecture

TradeX is built on a robust three-tier Model-View-Controller (MVC) architecture using the MERN stack. The React client handles user interactions and UI rendering, the Express backend executes business logic and route protection, and MongoDB Atlas handles persistent document storage via Mongoose schemas.

3.1 System Architecture Flow



3.2 Frontend Architecture

The React frontend is built using Vite for fast bundling and hot module replacement. It manages client-side routing, global state via GeneralContext.jsx, API communication via a pre-configured Axios instance, and renders interactive stock charts and real-time portfolio dashboards.

3.3 Backend Architecture

The backend is structured strictly following the MVC design pattern. Express routes (e.g., userRoute.js, stockRoute.js, orderRoute.js, transactionRoute.js) receive incoming REST requests and forward them to corresponding controllers (userController.js, stockController.js, orderController.js, transactionController.js). Middleware functions perform JWT validation and role authorization before controller execution.

3.4 Database Architecture

Data persistence is powered by MongoDB Atlas. Mongoose schemas define the structures for Users (credentials, balance, roles), Stocks (symbol, company details, market prices), Orders (type, status, quantity, user reference), and Transactions (historical logs).

3.5 Deployment Architecture

- **Frontend Deployment:** Hosted on Vercel (connected to GitHub repository for automated builds).
- **Backend Deployment:** Hosted on Render as a Node.js web service.
- **Database Deployment:** Hosted on MongoDB Atlas cloud database cluster.

4. Setup Instructions

4.1 Prerequisites

- **Node.js:** Version 18.x or higher installed on local machine.
- **npm:** Node Package Manager (comes bundled with Node.js).
- **MongoDB Atlas:** Active cloud MongoDB instance connection string.
- **Git:** Installed for repository cloning and version control.
- **Code Editor:** VS Code recommended.

4.2 Installation

Step 1: Clone Repository & Create Directory Structure

```
git clone <repository-url>
cd TradeX
mkdir client server
```

Step 2: Server Setup & Dependencies

```
cd server
npm init -y
npm install express mongoose dotenv cors jsonwebtoken bcryptjs
```

Step 3: Client Setup & Dependencies

```
cd ../client
npm create vite@latest . -- --template react
```

npm install
npm install axios react-router-dom lucide-react

4.3 Environment Variables

Server Environment Variables (server/.env):

Variable	Purpose
PORT	8000 (Port on which Node/Express server listens)
MONGO_URI	your_mongodb_atlas_connection_string
JWT_SECRET	your_jwt_secret_key

Client Environment Variables (client/.env):

Variable	Purpose
VITE_API_URL	http://localhost:8000 (Base API URL for local development or deployed server URL)

5. Folder Structure

The TradeX repository is cleanly partitioned into Client and Server subdirectories:

5.1 Client Folder (React + Vite)

```
client/
├── public/
├── src/
│   ├── components/
│   │   ├── axiosInstance.js    # Pre-configured Axios instance with BaseURL
│   │   ├── Login.jsx           # User login component
│   │   ├── Navbar.jsx          # Top navigation bar component
│   │   └── Register.jsx        # User registration component
│   ├── context/
│   │   └── GeneralContext.jsx # React Context for global auth state
│   ├── pages/
│   │   ├── Admin.jsx           # Admin main dashboard page
│   │   ├── AdminStockChart.jsx # Admin stock chart inspection page
│   │   ├── AllOrders.jsx       # Admin overview of all user orders
│   │   ├── AllTransactions.jsx # Admin overview of all trading transactions
```

```
│   │   ├── History.jsx        # User historical orders and transactions
│   │   ├── Home.jsx           # User main home dashboard page
│   │   ├── Landing.jsx        # Public landing page
│   │   ├── Portfolio.jsx      # User virtual portfolio and holdings page
│   │   ├── Profile.jsx        # User profile and account management
│   │   ├── StockChart.jsx     # Detailed stock price chart page
│   │   └── Users.jsx          # Admin user management page
│   ├── App.jsx                # Client routes and layout wrapper
│   └── main.jsx               # Vite entry point
├── .env                       # Frontend environment variables
├── index.html                 # HTML template
└── package.json               # Frontend dependencies & scripts
```

5.2 Server Folder (Node.js + Express)

```
server/
├── config/
│   └── db.js                  # Mongoose MongoDB connection setup
├── controllers/
│   ├── orderController.js     # Logic for buy/sell order execution & management
│   ├── stockController.js     # Logic for stock listing, price feeds & charts
│   ├── transactionController.js # Logic for recording & fetching transactions
│   └── userController.js      # Logic for registration, login, balance & profile
├── middlewares/
│   └── authMiddleware.js      # JWT verification & role authorization check
├── models/
│   ├── orderSchema.js         # Mongoose schema for orders
│   ├── stockSchema.js         # Mongoose schema for stocks and prices
│   ├── transactionModel.js    # Mongoose schema for transaction logs
│   ├── userModel.js           # Mongoose schema for user accounts
│   └── Schemas.js             # Centralized schema exports
├── routes/
│   ├── orderRoute.js          # REST API endpoints for orders
│   ├── stockRoute.js          # REST API endpoints for stock data
│   ├── transactionRoute.js    # REST API endpoints for transactions
│   └── userRoute.js           # REST API endpoints for auth & user operations
├── .env                       # Backend environment variables
├── index.js                   # Main server entry point & middleware setup
└── package.json               # Backend dependencies & scripts
```

6. Running the Application

6.1 Running Backend Server

1. Open terminal and navigate to server directory:
   cd server
2. Install dependencies (if not already installed):

npm install
3. Start backend server using nodemon or node:
npm start (or nodemon index.js)
Server will start on http://localhost:8000 and connect to MongoDB Atlas.

6.2 Running Frontend Client

1. Open new terminal window and navigate to client directory:
cd client
2. Install dependencies:
npm install
3. Launch Vite development server:
npm run dev
Application will be accessible locally at http://localhost:5173.

7. API Documentation

TradeX exposes RESTful JSON endpoints categorized by functional module:

7.1 Authentication & User APIs

Method	Endpoint	Description	Access
POST	/api/users/register	Register a new user account	Public
POST	/api/users/login	Authenticate user & return JWT token	Public
GET	/api/users/profile	Fetch logged-in user profile & balance	Authenticated
PUT	/api/users/profile	Update profile or deposit/withdraw funds	Authenticated

7.2 Stock & Market Data APIs

Method	Endpoint	Description	Access
GET	/api/stocks/	Fetch all US stock listings	Authenticated
GET	/api/stocks/:id	Fetch specific stock details & price chart	Authenticated

POST	/api/stocks/	Create a new stock listing	Admin
PUT	/api/stocks/:id	Update stock price or exchange data	Admin

7.3 Order & Paper Trading APIs

Method	Endpoint	Description	Access
POST	/api/orders/	Place market buy or sell stock order	Trader
GET	/api/orders/my-orders	Fetch orders for authenticated user	Trader
GET	/api/orders/	Fetch all system orders across platform	Admin

7.4 Transaction APIs

Method	Endpoint	Description	Access
GET	/api/transactions/my-transactions	Fetch transaction history log	Trader
GET	/api/transactions/	Fetch global transaction logs	Admin

8. Authentication & Security

8.1 Password Encryption

User passwords are encrypted before database insertion using bcryptjs salted hashing (salt rounds = 10). Plaintext passwords are never stored.

8.2 JWT Authentication Flow

Upon successful authentication via POST /api/users/login, the server returns a signed JSON Web Token containing the user's ID and role. The client includes this token in the HTTP Authorization header (Bearer token) for all protected API calls.

8.3 Role-Based Authorization Middleware

The authMiddleware.js inspects request headers, decodes the JWT, and enforces permission checks. Administrative endpoints (e.g. creating stocks, viewing all user accounts) are restricted exclusively to users with the 'Admin' role.

9. User Interface & Page Workflows

- **Landing Page (Landing.jsx):** Public welcoming hero section introducing platform features with quick links to Register and Login.
- **Registration & Login (Register.jsx, Login.jsx):** Clean form interfaces with input validation and instant state update on login.
- **Trader Dashboard (Home.jsx):** Central hub summarizing virtual funds, account portfolio balance, and stock market quick access.
- **Stock Listings & Charts (StockChart.jsx):** Interactive stock detail pages displaying live ticker prices and historical performance trends.
- **Portfolio Management (Portfolio.jsx):** Holdings dashboard showing owned stocks, quantity, average buy price, current value, and live P&L.
- **Transaction Log (History.jsx):** Comprehensive table listing all past buy and sell operations with exact execution timestamps.
- **Admin Console (Admin.jsx, Users.jsx, AllOrders.jsx):** Control panel allowing administrators to manage users, inspect system-wide orders, and update stock feeds.

10. Testing Strategy & Verification

10.1 Backend API Testing

All Express REST API endpoints were systematically verified using Postman to confirm valid status codes (200, 201), correct error handling (400, 401, 403, 500), and proper database mutations.

10.2 Frontend & State Testing

React components were manually tested across Google Chrome and Mozilla Firefox browsers. Verified state persistence in GeneralContext.jsx during page refreshes and navigation transitions.

11. Deployment Details

Layer	Service Provider	Deployment Target
Frontend Client	Vercel	Vercel Cloud Deployment
Backend Server	Render	Render Web Service Deployment
Cloud Database	MongoDB Atlas	Managed Cloud MongoDB Cluster

12. Known Issues & Limitations

- **Internet Dependency:** Requires an active internet connection to communicate with MongoDB Atlas cloud database and external price feeds.
- **Free Hosting Spin-Up Delays:** Render free-tier instances may experience a 30-50 second cold-start latency on initial backend API calls after idle periods.

13. Future Enhancements

- **Real-Time WebSockets:** Integrate Socket.io to stream real-time tick-by-tick stock price updates directly to the frontend without polling.
- **Advanced Technical Indicators:** Add MACD, RSI, and Bollinger Bands charting overlays for advanced technical stock analysis.
- **Leaderboard & Social Trading:** Implement a public trader leaderboard ranking users by portfolio return percentage.